SafeQuant: Experiment Log & Research Journey

Research Question

How does post-training quantization damage safety alignment in LLMs, and can we localize where the damage occurs?

1. Models & Quantizers

Models

Alias	Model	Architecture	Params	Notes
`llama7b`	Meta-Llama-3.1-8B-Instruct	LLaMA 3.1	8.03B	Safety-aligned (RLHF)
`mistral7b-v02`	Mistral-7B-Instruct-v0.2	Mistral	7.24B	Less aligned than Llama
`qwen7b`	Qwen2.5-7B-Instruct	Qwen 2.5	7.62B	Cross-architecture validation

Quantizers Tested

Quantizer	Venue	Bits	Method	Layer-swap compatible?
INT8 (LLM.int8)	NeurIPS 2022	8	Per-channel absmax	■ Yes
NF4 (QLoRA)	NeurIPS 2023	4	Normal float lookup	■ Yes
GPTQ	NeurIPS 2022	4/3/2	One-shot weight-optimal	■ Yes
AQLM	ICML 2024	2	Learned codebooks	■ Yes
AWQ	MLSys 2024	4	Activation-aware scaling	■ No (see §4)

2. Experiment 1: Read-Side vs Write-Side Restoration

Methodology

Transformer linear projections fall into two categories:

- **Read-side** (information intake): `q_proj`, `k_proj`, `v_proj`, `gate_proj`, `up_proj`
- **Write-side** (information output): `o_proj`, `down_proj`

Protocol:

1. Load FP32 model → extract all projection weights

2. Load quantized model → generate responses (baseline)
3. Replace write-side projections with FP32 weights → generate (write-restored)
4. Replace read-side projections with FP32 weights → generate (read-restored)
5. Replace both → generate (both-restored, sanity check ≈ FP32)
6. Classify all responses with HarmBench judge → compute ASR

Metrics:

- ASR (Attack Success Rate): % of harmful prompts where model complied
- ARC (ARC-Challenge): Reasoning capability (50 questions)
- Recovery %: How much of the quantization damage is reversed by restoration

Results

Llama-3.1-8B-Instruct

Quantizer	FP32 ASR	Q ASR	Damage	Write Rec	Read Rec	Verdict
INT8	42%	45%	+3pp	33%	**133%**	**Read-side**
NF4	42%	39%	-3pp	**100%**	33%	Write-side
GPTQ-4	42%	40%	-2pp	-100%	**-200%**	**Read-side**
GPTQ-3	42%	14%	-28pp	**139%**	-21%	Write-side
GPTQ-2	42%	17%	-25pp	-40%	**-56%**	**Read-side**
AQLM-2	42%	63%	+21pp	67%	**90%**	**Read-side**

Mistral-7B-Instruct-v0.2

Quantizer	FP32 ASR	Q ASR	Damage	Write Rec	Read Rec	Verdict
INT8	74%	75%	+1pp	—	—	Minimal damage
GPTQ-4	74%	72%	-2pp	100%	**250%**	**Read-side**
GPTQ-3	74%	76%	+2pp	**300%**	50%	Write-side
GPTQ-2	74%	5%	-69pp	77%	**80%**	**Read-side**
AQLM-2	74%	64%	-10pp	70%	**80%**	**Read-side**

Qwen2.5-7B-Instruct

Quantizer	FP32 ASR	Q ASR	Damage	Write Rec	Read Rec	Verdict
INT8	55%	62%	+7pp	**100%**	14%	Write-side
GPTQ-3	55%	58%	+3pp	-100%	**100%**	Mixed
GPTQ-2	55%	9%	-46pp	13%	**98%**	**Read-side**

(NF4, GPTQ-4 pending)

Key Finding

Read-side collapse is the dominant pattern — 9 out of 14 valid experiments show read-side matrices carrying more safety-critical information. This is consistent with the mechanistic interpretability view that attention query/key/value projections encode the model's "understanding" of input safety, while output projections merely route processed information.

Sanity Check: Both-Restored ≈ FP32

In all experiments, restoring both read and write sides recovers ASR and ARC to near-FP32 levels, confirming:

- Quantization damage is entirely in linear projection weights
- LayerNorms, embeddings, and attention mechanisms are not damaged
- The damage is reversible — safety circuits are distorted, not destroyed

3. Experiment 2: Dimension-Level Activation Patching

Motivation

If read-side matrices carry safety information, can we identify *which specific dimensions* within those matrices are most safety-critical?

Methodology

Instead of restoring entire weight matrices, restore individual columns (dimensions) selected by targeting metrics:

- **V2**: `|refusal_direction[j]| × quantization_error[j]`
- **V3**: `|refusal_direction[j]| × quantization_error[j] × mean_activation[j]`
- **Random**: Random dimension selection (control)

Tested at 5%, 15%, 25%, 50% of dimensions restored.

Results (Llama / AQLM-2bit, damage = +21pp)

% Restored	V2 ASR	V3 ASR	Random ASR	FP32=42%, AQLM2=63%
5%	63%	61%	63%	Minimal effect
15%	58%	56%	55%	All similar
25%	**52%**	57%	**69%** ↑	Random HURTS
50%	56%	**51%**	61%	V3 best

Problems Identified

1. **Non-monotonic behavior**: More restoration doesn't always help (V2: 25%→52% but 50%→56%)
2. **Frankenstein effect**: Mixing 2-bit and 16-bit columns within the same weight matrix creates incoherent representations
3. **Random at 25% = 69% ASR**: Randomly restoring wrong dimensions makes the model LESS safe than both baseline and FP32
4. **Model-dependent**: V3 works on Llama but destroys Mistral's ARC (62%→44%)

Conclusion

Binary dimension-level swapping is fundamentally flawed due to the Frankenstein problem. Led to Experiment 3 (interpolation).

4. Failure: AWQ Incompatibility

What We Tried

1. Custom AWQ INT4 unpacking (pure-torch dequantization)
2. Identity matrix forward pass for weight extraction
3. AWQ scaling transformation (multiply/divide by `module.scales`)
4. Raw FP32 weight replacement without transformation

What We Found

Debug test (definitive proof):

```
Baseline AWQ cosine with FP32:        0.9957  (works perfectly natively)
Replace 1 layer with FP32:            0.9257  (degraded)
Replace all q_proj (32 layers):       0.7115  (badly broken)
Replace ALL 224 projections with FP32: 0.7411  (should be ~1.0 but isn't!)
```

Replacing ALL linear layers with FP32 gives only 0.74 cosine similarity — proving AWQ modifies **non-linear layers** (LayerNorms) during calibration.

Root Cause

AWQ's activation-aware scaling works by:

1. Finding per-channel importance factors `s`
2. Scaling weights: `W_awq = W × s`
3. **Absorbing `1/s` into adjacent LayerNorms**

The LayerNorms and linear weights are **co-dependent**. Replacing linear weights without also replacing their matched LayerNorms creates a mismatch.

Why Other Quantizers Work

GPTQ, NF4, INT8, and AQLM only modify the linear weights — LayerNorms remain identical to FP32. So layer-swap restoration works cleanly.

Publishable Finding

> AWQ's cross-layer activation scaling makes it uniquely incompatible with mechanistic interpretability methods that assume layer-wise independence. This is a methodological consideration for future safety evaluation of quantized models.

5. Experiment 3: Interpolation Restoration (In Progress)

Motivation

The binary dimension swap (Experiment 2) suffers from the Frankenstein problem. Instead of restoring a few dimensions fully, what if we improve ALL dimensions partially?

Methodology

```
W_corrected = W_quantized + α_j × (W_fp32 - W_quantized)
```

Where α_j varies by method:

- **Uniform**: Same α for all dimensions (α = 0.1, 0.25, 0.5, 0.75, 1.0)
- **Safety-weighted**: $\alpha_j \propto$ `|refusal_dir[j]| × quant_error[j] × activation[j]`
- **Error-only**: $\alpha_j \propto$ `quant_error[j]` (ablation)

Why This Should Work

- No sharp 2-bit $\leftrightarrow$ 16-bit transitions within a weight matrix
- Every column stays at a similar effective precision
- Safety-critical dimensions get more correction, but smoothly
- The weight matrix remains internally coherent

Expected Results

- Safety-weighted at α=0.25 should beat uniform at α=0.5 (targeted is more efficient)
- Monotonic improvement with increasing α (no Frankenstein artifacts)
- ARC should not crash (no mixed-precision coherence issues)

Status

Job queued (pending priority).

6. Infrastructure Challenges

auto_gptq Kernel Issues

- Prebuilt wheel lacked CUDA extensions → 10× slower inference
- Triton backend crashed (`PY_SSIZE_T_CLEAN` — triton 3.3.1 incompatible with auto_gptq 0.7.1)
- **Fix**: Skipped auto_gptq entirely, used transformers native `GPTQConfig(use_exllama=False)`

Disk Quota

- Ocean storage (shared 2.1TB) hit 100% capacity
- Cleaned ~45GB of unused model caches (QTIP, old AQLM variants, Gemma, Qwen-0.5B)
- Redirected pip cache to `/tmp`

AWQ Dequantization Saga

- `dequantize_gemm` raised `StopIteration` (exllama-v2 repacked format)
- `next(module.parameters())` raised `StopIteration` (buffers, not parameters)
- Pure-torch INT4 unpacking produced wrong weights (wrong bit ordering)
- Identity matrix forward pass failed (kernel bug)
- **Final conclusion**: AWQ is fundamentally incompatible (not a code bug)

Package Version Conflicts

- `--target` pip install corrupted conda env (numpy 1.26 vs 2.4 mismatch)
- `aqlm[gpu]` install upgraded torch 2.7→2.11, had to roll back
- C4 dataset cache hash mismatch after datasets library update

7. Current Status

Complete Results Matrix

	INT8	NF4	GPTQ-4	GPTQ-3	GPTQ-2	AQLM-2
Llama-3.1-8B	■	■	■	■	■	■
Mistral-7B-v0.2	■	■	■	■	■	■
Qwen2.5-7B	■	■	■	■	■	N/A

■ = submitted, pending

Activation Patching Results

Model / Quantizer	Status
llama7b / gptq4	■ Done
llama7b / aqlm2	■ Done
mistral7b-v02 / aqlm2	■ Done

Interpolation Restoration

Model / Quantizer	Status
llama7b / aqlm2	■ Queued
mistral7b-v02 / aqlm2	■ Queued

8. File Structure

```
safequant/
■■■ config.py                          # Model & checkpoint configs
■■■ core/
■   ■■■ model_loader.py                # Unified loader (all quantizers)
■   ■■■ datasets.py                    # WildJailbreak, ARC
■   ■■■ evaluation.py                  # ASR/ARC measurement
■   ■■■ refusal_direction.py           # Refusal direction extraction
■■■ diagnostics/
■   ■■■ readwrite_restoration.py       # Exp 1: Read vs write restoration
■   ■■■ activation_weighted_patch.py   # Exp 2: Dimension-level patching
■   ■■■ interpolation_restore.py       # Exp 3: Smooth interpolation
■■■ scripts/
■   ■■■ rw_*.sbatch                    # Readwrite restoration jobs
■   ■■■ interpolation_restore.sbatch   # Interpolation job
■   ■■■ activation_patch_aqlm2.sbatch  # Activation patching job
■   ■■■ quantize_*.py/.sbatch          # Quantization utilities
■   ■■■ download_models.py             # Model downloader
■■■ outputs/                           # Result JSONs
■■■ _trash/                            # Archived old files
```

8.1 Log Files Reference

All logs are in `scripts/logs/`. Archived logs in `scripts/logs/_old/`.

Readwrite Restoration Results

Log file	Contents	Models	Quantizers
`nf4_gptq3_2_awq_llama_qwen4_aqlm2.out`	Multi-quantizer run for Llama	llama7b	nf4, gptq4, gptq3, gptq2, awq (invalid), aqlm2
`int8_all_models.out`	INT8 readwrite for all 3 models	llama7b, mistral7b-v02, qwen7b	int8
`aqlm2_llama_mistral.out`	AQLM 2-bit readwrite	llama7b, mistral7b-v02	aqlm2
`gptq4_3_2_mistral_v02.out`	Mistral v0.2 GPTQ sweep	mistral7b-v02	gptq4, gptq3, gptq2

Log file	Contents	Models	Quantizers
`gptq3_2_qwen7b.out`	Qwen7b GPTQ 3-bit and 2-bit	qwen7b	gptq3, gptq2
`qwen_gptq2_no_gibberish.out`	Qwen GPTQ-2bit (fixed version)	qwen7b	gptq2
`rw_missing_*.out`	Missing experiments fill-in	mistral7b-v02, qwen7b	nf4, gptq4

Activation Patching Results

Log file	Contents	Models	Quantizers
`activation_patch_38639233.out`	Dim-level patching (V2 vs V3 vs random)	llama7b	gptq4
`activation_patch_aqlm2_39087583.out`	Dim-level patching with larger signal	llama7b, mistral7b-v02	aqlm2

Interpolation Restoration (Experiment 3)

Log file	Contents	Status
`interpolation_*.out`	Smooth interpolation (uniform/safety-weighted/error-only)	Pending in queue

AWQ Investigation

Log file	Contents	Key finding
`awq_not_working_reason.out`	AWQ debug test — replaced 1/32/224 layers	Cosine=0.7411 when ALL replaced (proves LayerNorm modification)

Utility / Quantization

Log file	Contents
`quantize_mistral_qwen7b_2_3.out`	GPTQ on-the-fly quantization (Mistral-v02 3/2bit, Qwen7b 3/2bit)

Where to find specific results

- **"Why did read-side collapse happen for Llama GPTQ-4?"** → `nf4_gptq3_2_awq_llama_qwen4_aqlm2.out`
- **"What was the AWQ cosine similarity debug?"** → `awq_not_working_reason.out`
- **"Did V3 activation targeting beat V2?"** → `activation_patch_aqlm2_39087583.out`
- **"How did INT8 affect Qwen safety?"** → `int8_all_models.out`
- **"What happened with random dimension restoration?"** → `activation_patch_aqlm2_39087583.out` (Llama 25%: random=69%, worse than baseline)

9. Key Conclusions So Far

1. **Read-side collapse is real**: Across models and quantizers, read-side matrices (q/k/v/gate/up) carry disproportionately more safety-critical information than write-side (o/down).

2. **Quantization damage is reversible**: Restoring both sides to FP32 recovers safety and reasoning to baseline levels. Safety circuits are distorted, not destroyed.

3. **AWQ is fundamentally different**: Its cross-layer scaling makes it incompatible with layer-wise interpretability methods. This is a property of AWQ's design, not a bug.

4. **Dimension-level restoration is noisy**: Binary swapping of individual columns creates Frankenstein weight matrices. Smooth interpolation (Experiment 3) is the promising alternative.

5. **Random restoration can harm safety**: Restoring random dimensions at 25% on AQLM-2bit increased ASR to 69% (worse than both baseline 63% and FP32 42%). Careless mixed-precision deployment is dangerous.

6. **Extreme quantization (2-bit) causes severe damage**: GPTQ-2bit on Mistral drops ASR from 74% to 5% (-69pp). AQLM-2bit handles 2-bit much better due to learned codebooks.